Model Version 1 Outline

1 Objective

The primary object is to use the pre-training plus fine-tuning paradigm, ontop of some kind of transformer architecture. The most obvious choice was to do win-prediction, hence the goal is win prediction.

Ultimately, this sub-project is about selecting and justifying an architecture (for the given task, which may not be well posed), and empirically learning about the effects of choices like optimizers, loss functions, regularization techniques, and subtle architecture adjustments. All is in the service of spurring on my own interest in the field.

2 Data

The data for this project comes from a public Deadlock data API. It comes in two forms:

1. End-of-Match - This is essentially a screenshot of the final scoreboard. This will include variables determined by the outcome of the game, like kills, death, items, and souls, but will also have element invariant of outcome, like character selection, lane assignment, and rank (elo).

2. Event Stream - This is literally the entire game. Every entities position at every tick of the game, every characters soul count, kills, deaths, assists, spatial position, ability cooldowns, etc. This is the ultra-high resolution spatial-temporal data that initial inspired me to consider this dataset.

These two datasets provide a nice form factor for the pre-training/fine-tuning paradigm. The first, End-of-Match data, is extremely small, and easy to get, providing the usual medium for pre-training. The second, Event Stream data, provide, again, an excellent structure for this paradigm; Extremely high resolution, expensive, well labeled data. However, the event stream data also has the bonus feature that the ultra-high detail of it allows for multimodal use of the data.

3 Architecture

The architecture I came up with is as follows:

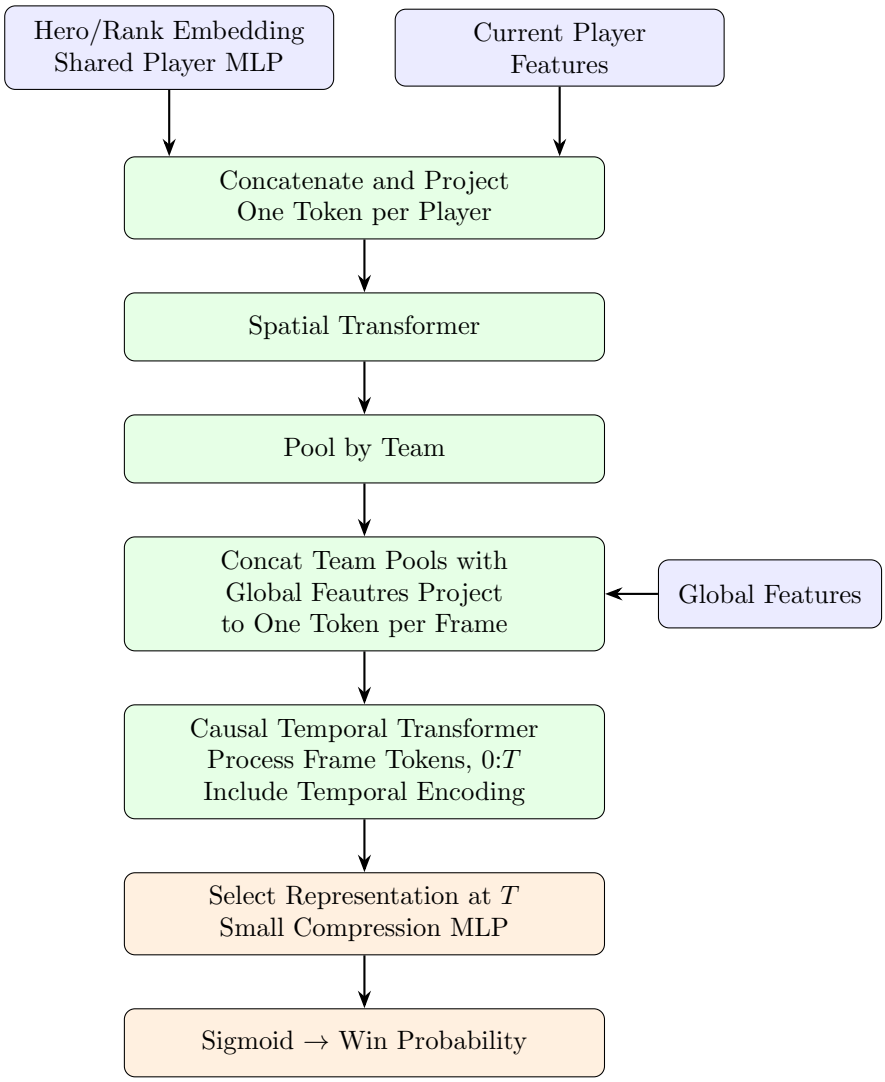


Figure 1: Win-prediction architecture. Shared player encoding and spatial attention are applied to each frame; temporal attention combines the observed history to predict the final match outcome.

Let $\mathcal{I}$ denote the set of heroes, e.g., $\mathcal{I} = \{\text{Abrams}, \text{Apollo}, \text{Warden}, \ldots\}$. For a given match, let $p \in \{1, \ldots, 12\}$ index players and $t \in \{0, \ldots, T\}$ index observed frames. Player p selects hero $i_p \in \mathcal{I}$ and has scaled pre-match rank $r_p \in \mathbb{R}_+$. Let $\mathbf{x}_{p,t} \in \mathbb{R}^F$ contain current player features, including team identity, position, kills and death, and let $\mathbf{g}_t \in \mathbb{R}^G$ contain global features like tower damage, urn count, elasped time. Let $[\mathbf{a}; \mathbf{b}]$ denote vector concatenation.

Component	**Purpose**	**Symbol & Structure**
Hero embedding	Learned hero lookup vector.	$e_i \in \mathbb{R}^{16}$ for each $i \in \mathcal{I}$
Static player MLP	Condition hero representation on rank.	$\mathbf{s}_p = f_{\text{static}}([\mathbf{e}_{i_p}; r_p]) \in \mathbb{R}^{32}$ where f_{static} is an MLP with ReLU layers MLP: $17 \to 32 \to 32$
Player projection	Combine static and current features.	$\mathbf{z}_{p,t} = \mathbf{W}_{\text{player}}[\mathbf{s}_p; \mathbf{x}_{p,t}] + \mathbf{b}_{\text{player}} \in \mathbb{R}^{64}$
Spatial transformer	Contextualize players within a frame.	$\mathbf{U}_t = \text{Spatial}(\mathbf{Z}_t) \in \mathbb{R}^{12\times 64}$; 1 block, 4 heads, feed-forward with width 128.
Team pooling	Summarize each team's players.	$\mathbf{a}_{k,t} = \frac{1}{6}\sum_{p\in\mathcal{P}_k} \mathbf{u}_{p,t} \in \mathbb{R}^{64}$, where $\mathcal{P}_k$ is team k's player set.
Frame projection	Combine team summaries and global state.	$\mathbf{v}_t = \mathbf{W}_{\text{frame}}[\mathbf{a}_{1,t}; \mathbf{a}_{2,t}; \mathbf{g}_t] + \mathbf{b}_{\text{frame}} \in \mathbb{R}^{64}$.
Temporal encoding	Represent frame order.	$\tilde{\mathbf{v}}_t = \mathbf{v}_t + \mathbf{q}_t$, where $\mathbf{q}_t \in \mathbb{R}^{64}$ is a positional encoding.
Temporal transformer	Represent observed match history.	$\mathbf{h}_T = \text{Temporal}(\tilde{\mathbf{v}}_0, \ldots, \tilde{\mathbf{v}}_T)_T \in \mathbb{R}^{64}$; 2 blocks, 4 heads, feed-forward with width 128.
Win head	Predict team 1's win probability.	$\hat{p}_T = \sigma(f_{\text{win}}(\mathbf{h}_T))$; where f_{win} is an MLP with ReLU layers $64 \to 32 \to 1$

Table 1: Proposed architecture. $\mathbf{Z}_t$ stacks player tokens as rows; $\mathbf{u}_{p,t}$ is player p's spatial output.

Now with the architecture figured out, how to train it becomes the next major question.

4 Training

In order to meet the goals of this sub-project, and take advantage of all the available data, this is the following training procedure I have come up with for this model. Note that numerical values for parameters are intial quantities and will likely be adjusted during training.

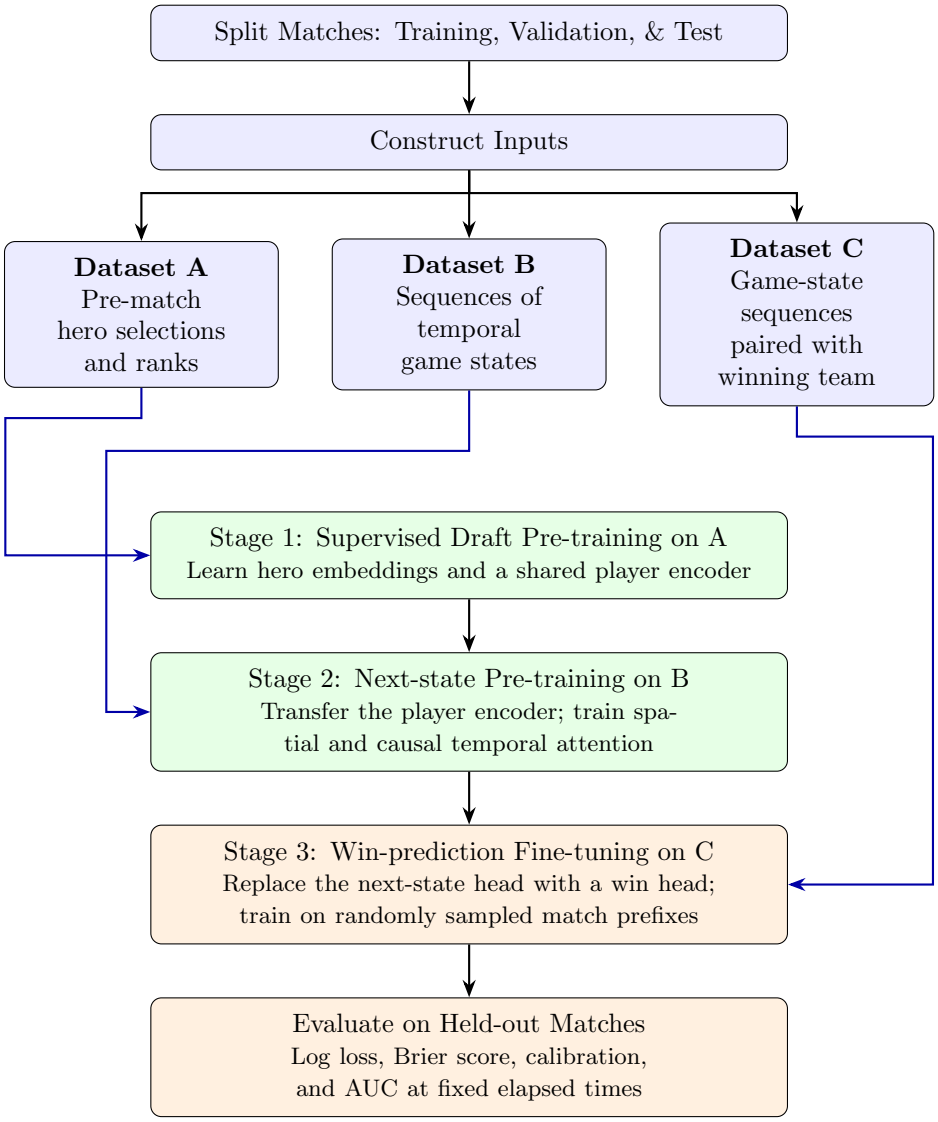


Figure 2: Proposed training procedure. Blue arrows supply each stage with its corresponding dataset; black arrows between training stages indicate parameter and data transfer. Datasets B and C may use the same trajectories, with different training targets.

using this high-level overview, I now consider each step in more detail.

4.1 Pre-Train 1 - Static Embeddings

The goal of this pre-training stage is to learn hero embeddings and a shared MLP that combines hero identity with pre-match rank. These components produce static player representations that are transferred to the subsequent training stages.

4.1.1 Architecture

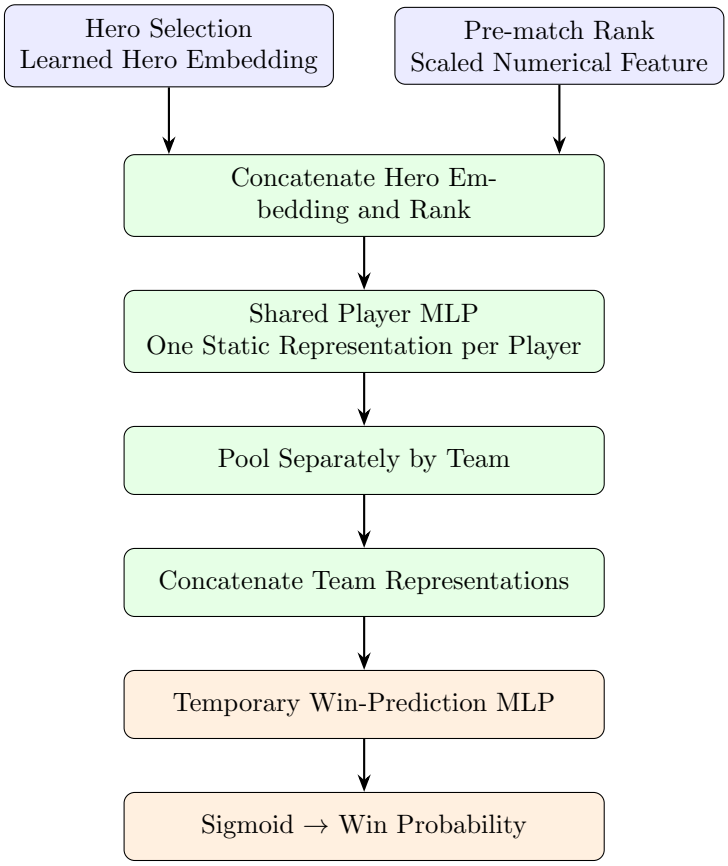


Figure 3: Draft-based pre-training architecture. Hero embeddings and the shared player MLP are trained using match outcomes and transferred to Stage 2. Everything after the pooling is discarded.

4.1.2 Loss function

Since this is a win-prediction task, I use binary cross-entropy between prediect probability that team 1 wins and the observed outcome. For the same of numerical stability, loss is computed directly from the output logit using `BCEWithLogitsLoss`.

4.1.3 Optimizer and learning rate

I use AdamW with an initial learning rate of 10^{-3} and weight decay of 10^{-4}. These are starting values, an may be adjusted later based on validation loss.

4.1.4 Batch size and training duration

I use mini-batches of 64 matches and will train for at most 100 epochs, shuffling the training matches at each epoch. I retain the checkpoint with the lowest validation binary cross-entropy.

4.1.5 Regularization and stopping criteria

I halt if the validation loss does not improve for 10 consecutive epochs. Initially, I do not use dropouts but this may change if training loss continues to decrease while validation loss worsens.

Similarly, I initially do not use weight decay, but in the event of overfitting, I will.

4.2 Pre-Train 2 - Temporal Dyanmics

4.3 Fine-Tune - Win Prediction

5 Evaluation

6 Open Questions

This architecture incorporates all the targeted, non-mutually exclusive, methods I wanted to exercise:

- **Embeddings** - Learned one-hot hero embeddings (like text) with numeric information, rank, to compose a composit embedding.
- **Transformers** - Using both spatial and temporal attention with different structures.
- **Parameter sharing** - Using the parameters from one model, or training, inside of another.
- **Residuals & Layer Normalizing** - Considerations for regularizing magnitudes, normalizing values between layers for training stability.
- **Causal Masking & Temporal Encoding** - Temporal encoding for game state flow, and causul masking to preserve temporal information flow.
- **Supervised Pre-Training** - Using high-quantity labeled data to learn embeddings.
- **Self-Supervised Pre-Training** - Using temporal game-state data to learn game dynamics.
- **Transfer Learning & Fine-Tuning** - Pre-trained components are reused as trained representation generation for win prediction.
- **Random-Prefix Training** - Randomly cut temporal observations to train model prediction outs over variable time horizons.